Small Language Models for Marine Hydrodynamics Problem Solving using RLPT

Thesis to be submitted in the partial fulfillment of the requirements for the degree

MASTER OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE

by

Raghuveer Patil
(21NA3AI42)

Under the guidance of

Prof. Prabhat Kumar Mishra

Department of Artificial Intelligence
Indian Institute of Technology, Kharagpur
24th April, 2026

Department of Artificial Intelligence
Indian Institute of Technology,
Kharagpur
India – 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled **Small Language Models for Marine Hydrodynamics Problem Solving using RLPT**, submitted by **Raghuveer Patil** (Roll Number: 21NA3AI42) a postgraduate student of Department of Artificial Intelligence in partial fulfillment for the award of degree of Master of Technology. We hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in partial fulfillment for the Post Graduate Degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and has reached the standard needed for submission.

Supervisor
Department of Artificial Intelligence
Indian Institute of Technology, Kharagpur

Place: Kharagpur
Date:

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Prof. Prabhat Kumar Mishra, who introduced me to the exciting intersection of Reinforcement Learning and domain-specific language modelling. His profound expertise and insightful guidance have been instrumental in the progress and success of my research. I am also indebted to the faculty and staff of the Department of Artificial Intelligence, whose assistance has been invaluable throughout my academic journey. Their willingness to share knowledge and offer advice has greatly enriched my learning experience. Special thanks are due to my peers and colleagues who have provided me with support and collaboration. Their perspectives and feedback have been a source of encouragement and growth. Finally, and most importantly, my heartfelt thanks go to my family. Their unwavering support, endless patience, and belief in my abilities have been my strength and perseverance. This accomplishment is as much theirs as it is mine.

Abstract

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in general-purpose reasoning, yet these models frequently underperform on specialized engineering tasks requiring structured numerical reasoning and domain-specific knowledge. This thesis investigates the central question of whether GRPO-based RL post-training can bring a Small Language Model (SLM) to frontier LLM performance levels on domain-specific engineering problems — specifically, whether a 3-billion-parameter model fine-tuned on marine hydrodynamics can match or exceed a general-purpose frontier LLM on the same evaluation set.

We propose a multi-stage pipeline comprising synthetic dataset generation, Supervised Fine-Tuning (SFT), and GRPO-based RL post-training via Group Relative Policy Optimization (GRPO). Source materials comprise the marine hydrodynamics textbooks of Newman (1977), Faltinsen (1993); MIT OpenCourseWare courses 2.20 and 2.29 (Massachusetts Institute of Technology 2005, 2003), taught respectively by Prof. Dick K. P. Yue and Prof. Jerome H. Milgram; and graduate examination papers for IIT Kharagpur subject NA20202 (**?**). These are processed through a text-extraction and OCR pipeline to produce approximately 563 text chunks. A synthetic question–answer generation system powered by the Groq API (LLaMA 3.3 70B) produces approximately 2,750 QA pairs spanning conceptual, numerical, and multiple-choice question types, each accompanied by a Chain-of-Thought reasoning trace.

The SFT stage fine-tunes a `Qwen2.5-3B-Instruct` model using LoRA adapters on an NVIDIA H100 NVL GPU. A significant debugging effort was required to resolve persistent NaN gradient failures traced to BF16 precision accumulation errors across multiple LoRA target modules. The final SFT model achieves a training loss of 0.9729 and evaluation token accuracy of 77.43%.

The GRPO stage introduces a physics-based reward function combining numerical correctness verification (via SymPy), MCQ letter-matching, and a format reward to

ensure non-zero intra-group advantage variance. Across five GRPO training runs — incorporating progressively refined reward functions, a clean data pipeline (SFT-2), a critical `max_new_tokens` fix, and expanded LoRA capacity — numerical accuracy improves from 27.60% (leak-free SFT-2 baseline) to 38.91%, and MCQ accuracy improves from 71.82% to 85.45%. Critical implementation challenges including the zero-gradient problem, TRL API incompatibilities, and a truncation bug corrupting the reward signal are documented and resolved.

To contextualise performance, the evaluation set is also presented to a frontier LLM (Gemini) via its web interface. The domain-fine-tuned SLM achieves **38.91% numerical accuracy** and **80.00% MCQ accuracy**, compared to Gemini's 37.10% and 78.18% respectively — demonstrating that GRPO-based RL post-training successfully brings the SLM to and slightly beyond frontier LLM performance on this domain-specific benchmark. Both models exhibit an identical formula-selection failure pattern, confirming that the remaining accuracy ceiling is a data representation limitation rather than a model capacity issue.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Recent developments in Large Language Models (LLMs) have demonstrated remarkable capabilities in reasoning, coding, and natural language understanding. Models with hundreds of billions of parameters can perform sophisticated multi-step reasoning and generate coherent, contextually appropriate responses across diverse domains. However, these models are computationally expensive and often inefficient for domain-specific tasks requiring structured numerical reasoning and deep subject-matter expertise.

Recent research indicates that Small Language Models (SLMs) with 1–8 billion parameters, when fine-tuned on specialized datasets, can match or outperform larger models on specific tasks while requiring significantly lower computational resources (Hu et al. 2025). Surveys of recent work confirm that task-specific SLMs frequently rival or exceed larger models on focused benchmarks such as reasoning, structured output generation, and domain-specific question answering (Red Hat AI 2024).

This thesis explores the use of SLMs trained with GRPO-based RL post-training to solve marine hydrodynamics problems. The **central goal** is to determine whether a small model with 3 billion parameters can be brought to frontier LLM performance levels on domain-specific hydrodynamics questions through GRPO-based RL post-training — leveraging synthetic dataset generation and physics-based reward verification to enable reinforcement learning in a data-scarce domain. This is motivated by the broader question in the field: can GRPO-based RL post-training compensate for the knowledge gap between a small fine-tuned model and a much larger general-purpose LLM?

1.2 Problem Statement

Hydrodynamics is a specialised engineering domain involving problems related to potential flow theory, wave–body interactions, ship hydrodynamics, boundary layer theory, and fluid forces and hydrodynamic coefficients. While general-purpose LLMs possess broad knowledge, they often perform poorly on specialised engineering problems requiring structured reasoning and equation-based solutions.

Three key challenges motivate this work:

1. **Lack of labelled datasets:** Very few publicly available question–answer datasets exist for marine hydrodynamics. Unlike natural language understanding or code generation, this domain has no established benchmark.

2. **Requirement for numerical and symbolic reasoning:** Many problems require equation derivations, numerical calculations, and multi-step reasoning that go beyond pattern matching.

3. **Evaluation difficulty:** Solutions must be verified using physical laws and equations rather than subjective evaluation, necessitating automated physics-based verification.

This thesis investigates whether a domain-specialised SLM trained using reinforcement learning with verifiable physics-based rewards can solve hydrodynamics problems more accurately than general-purpose approaches.

1.3 Research Objectives

The primary objectives of this project are:

1. **To develop a task-specific Small Language Model** for marine hydrodynamics problem solving.

2. **To create a hydrodynamics dataset** using synthetic data generation techniques from the textbooks and open course materials cited in Section 3.2.1.

3. **To design a physics-based reward verification system** for reinforcement learning that combines numerical comparison, symbolic equivalence checking, and structural format verification.

4. **To train the model using Supervised Fine-Tuning (SFT) followed by GRPO-based RL post-training**, establishing a complete two-stage alignment pipeline.

5. **To evaluate whether GRPO-based RL post-training can bring the SLM to frontier LLM performance levels** by comparing the final model against a general-purpose LLM (Gemini) on the same held-out evaluation set.

6. **To document the complete engineering process**, including all debugging challenges, failure modes, and their resolutions, to aid reproducibility.

1.4 Contributions

This thesis makes the following contributions:

- **Marine Hydrodynamics QA Dataset:** A synthetic dataset of approximately 2,750 question–answer pairs with Chain-of-Thought reasoning, spanning conceptual, numerical, and multiple-choice question types, generated from the textbooks of Newman (1977), Faltinsen (1993), MIT OpenCourseWare materials (Massachusetts Institute of Technology 2005, 2003), and IIT Kharagpur NA20202 examination papers (**?**).

- **Domain-Specific SLM Training Pipeline:** A complete, reproducible pipeline from text extraction and OCR through synthetic QA generation, SFT, and GRPO training, all operating on a single GPU server.

- **Physics-Grounded Reward Function:** A multi-component reward function for GRPO that combines numerical correctness verification (with graded partial credit), MCQ letter-matching, and format rewards to ensure effective policy gradient training.

- **Detailed Error Analysis:** Comprehensive documentation of critical debugging challenges including BF16 NaN gradient failures, TRL API incompatibilities, the GRPO zero-gradient problem, and a truncation bug affecting reward signals.

- **Empirical Results:** Evidence that five GRPO training runs improve numerical accuracy from 27.60% to 38.91% and MCQ accuracy from 71.82% to 85.45% over the leak-free SFT-2 baseline, with a systematic analysis of the numerical accuracy ceiling and its root cause.
- **LLM Baseline Comparison:** A direct comparison of the fine-tuned SLM against a frontier LLM (Gemini) on the same evaluation set, demonstrating that GRPO-based RL post-training successfully brings the 3B SLM to and slightly beyond frontier LLM performance on this domain-specific benchmark (38.91% vs. 37.10% numerical accuracy; 80.00% vs. 78.18% MCQ accuracy).

1.5 Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2** provides a literature review of Small Language Models, LLM alignment techniques, reinforcement learning for reasoning, and parameter-efficient fine-tuning methods.
- **Chapter 3** describes the methodology, covering the data pipeline from text extraction through synthetic QA generation, the SFT training process, and the GRPO reinforcement learning implementation.
- **Chapter 4** presents the experimental results across all training stages, including SFT convergence analysis, GRPO training dynamics across three runs, and evaluation metrics.
- **Chapter 5** provides a dedicated analysis of errors and debugging, documenting the NaN investigation, API incompatibilities, and reward-signal corruption.
- **Chapter 6** concludes the thesis and outlines future research directions.

Chapter 2

Literature Review

2.1 Introduction

The rapid progress in large language models has intensified the need for reliable methods to specialise these models for domain-specific tasks. Although modern LLMs demonstrate strong zero-shot and instruction-following capabilities, these behaviours do not naturally extend to specialised scientific and engineering domains that require precise numerical reasoning. This chapter reviews the foundational methods underlying this thesis, covering transformer architectures, supervised fine-tuning, reinforcement learning from human feedback, and parameter-efficient adaptation techniques.

2.2 Foundations of Transformer Models

Transformer architectures, first proposed by Vaswani et al. (2017), form the basis for nearly all state-of-the-art language models. Unlike recurrent or convolution-based models, transformers rely entirely on self-attention mechanisms to capture long-range dependencies, allowing them to scale to billions of parameters while maintaining efficient parallelism during training.

2.2.1 Transformer Architecture Overview

The standard decoder-only transformer, widely used in LLMs, consists of stacked blocks each containing multi-head self-attention (MHA), a feed-forward network (FFN), layer normalization, and residual connections.

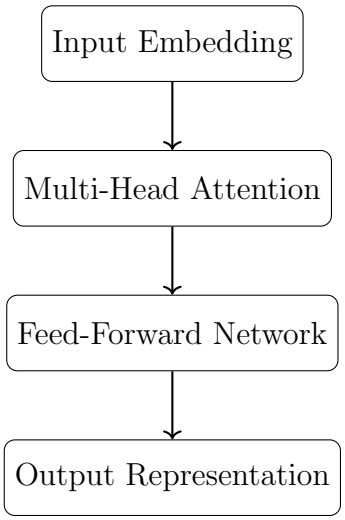

Figure 2.1: Decoder-only transformer block structure.

Multi-head attention computes weighted representations using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{2.1}$$

2.3 Supervised Instruction Tuning (SFT)

Supervised fine-tuning is typically the first step in adapting an LLM for a target domain. During SFT, the model is trained on high-quality instruction–response pairs that demonstrate desirable behaviours. Given input–output pairs (x, y), SFT minimizes cross-entropy loss:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y)}[\log \pi_\theta(y|x)]. \tag{2.2}$$

SFT establishes a foundational alignment layer, making the model's initial behaviour stable enough for more delicate preference-based or reinforcement-learning-based optimization. However, SFT alone cannot guarantee accurate numerical reasoning, which motivates the subsequent RL stage.

2.4 Reinforcement Learning from Human Feedback (RLHF)

RLHF has emerged as a key component of LLM alignment pipelines, used in models such as InstructGPT (Ouyang et al. 2022). RLHF introduces a reward model trained

on human preference data, which assigns a scalar reward to model outputs. The policy is then optimized via reinforcement learning to maximize expected reward.

2.4.1 KL-Regularized RL

To prevent the model from drifting too far from the reference policy, KL regularization is added:

$$\max_{\theta} \mathbb{E}\left[r_\phi(x, y) - \beta \, \mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})\right]. \tag{2.3}$$

2.4.2 Policy Gradient Foundations

RLHF ultimately relies on the policy gradient theorem (Sutton et al. 1999):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) A(s, a)], \tag{2.4}$$

where $A(s, a)$ is the advantage function.

2.5 RL-Based Alignment Methods

2.5.1 Proximal Policy Optimization (PPO)

PPO (Schulman et al. 2017) introduces a clipping mechanism to prevent large policy updates:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\left[\min\left(r_\theta A, \ \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)A\right)\right], \tag{2.5}$$

where $r_\theta = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. PPO is memory-intensive, requiring three models simultaneously (policy, reference, value), making it impractical for resource-constrained settings.

2.5.2 Group Relative Policy Optimization (GRPO)

GRPO was introduced in the DeepSeekMath work (Shao et al. 2024). It removes the need for a value network by normalizing advantages across groups of samples:

$$A_i^G = A_i - \frac{1}{|G|} \sum_{j \in G} A_j. \tag{2.6}$$

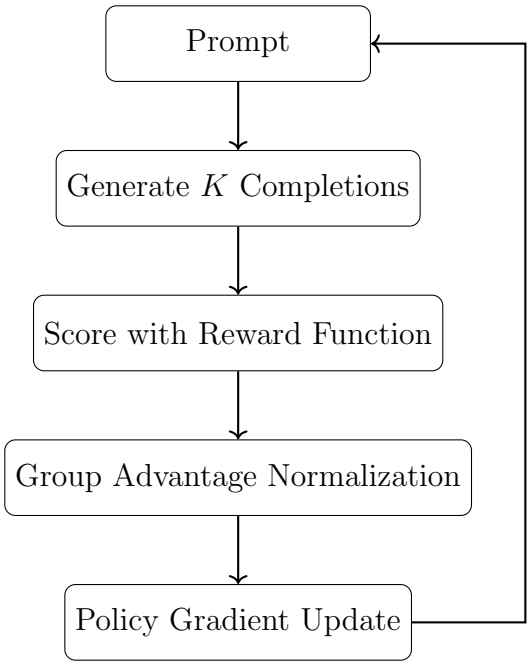

Figure 2.2: GRPO training loop: group-normalized advantages eliminate the value network.

By eliminating the value model, GRPO reduces VRAM requirements and improves training stability, making it particularly suited for low-resource reinforcement learning. GRPO is central to the RLPT approach used in this thesis.

2.6 Small Language Models

Recent research has demonstrated that task-specialised small models (1–8B parameters) can match or exceed larger models on structured tasks (Hu et al. 2025). SLMs fine-tuned for structured output generation have achieved over 98% formatting accuracy, outperforming larger models on constrained decoding tasks (Red Hat AI 2024). These findings suggest that specialised SLMs can be highly effective in narrow domains such as engineering and scientific problem solving.

2.7 Low-Rank Adaptation (LoRA)

LoRA (Hu et al. 2022) freezes base model weights and introduces trainable low-rank decomposition matrices:

$$W' = W + BA, \tag{2.7}$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. This dramatically reduces the number of trainable parameters while preserving the capacity for task-specific adaptation.

2.8 The Qwen Model Family

Qwen2.5 (Yang et al. 2024) is an open-source language model series developed by Alibaba. The `Qwen2.5-3B-Instruct` variant used in this thesis features a decoder-only transformer with RoPE positional embeddings, SwiGLU feed-forward networks, and has been instruction-tuned on high-quality supervised data. Its 3-billion parameter size provides a strong balance between capability and computational efficiency, making it well-suited for domain-specific fine-tuning on a single GPU.

2.9 Summary

This chapter reviewed the technical foundations underlying this thesis. We covered transformer architectures, supervised fine-tuning, RLHF and its policy optimization variants (PPO, GRPO), the emerging capabilities of Small Language Models, and parameter-efficient fine-tuning via LoRA. Together, these methods provide the conceptual and practical framework for the domain-specific training pipeline implemented in subsequent chapters.

Chapter 3

Methodology

3.1 Introduction

This chapter describes the complete methodology used to build a physics-aware Small Language Model for marine hydrodynamics. The pipeline spans five stages: data acquisition and text extraction, synthetic QA dataset generation, data validation and cleaning, Supervised Fine-Tuning (SFT), and Group Relative Policy Optimization (GRPO). The goal is to transform a general-purpose 3B parameter language model into a domain specialist capable of solving conceptual, numerical, and multiple-choice hydrodynamics problems.

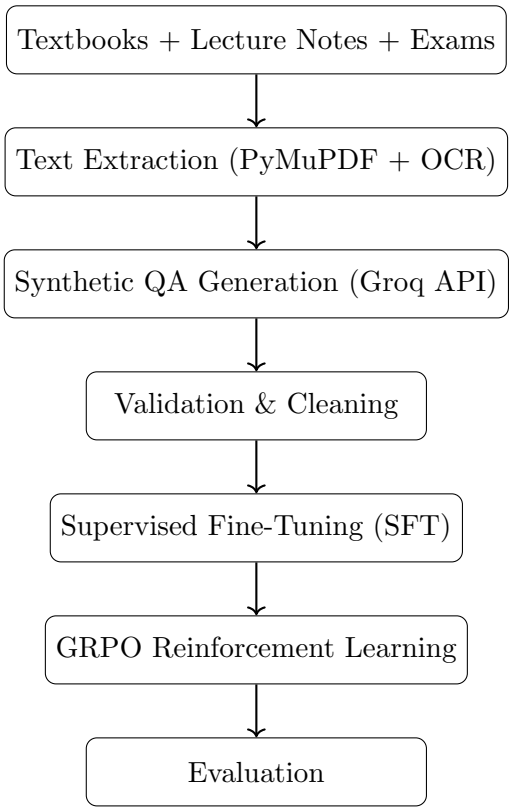

Figure 3.1: End-to-end pipeline for the Marine Hydrodynamics SLM.

3.2 Data Acquisition and Text Extraction

3.2.1 Source Materials

The training data is derived from the following sources (all cited in the bibliography):

- **Textbooks:** *Marine Hydrodynamics* (Newman 1977) (digital PDF, primary source) and *Sea Loads on Ships and Offshore Structures* (Faltinsen 1993) (OCR-scanned).
- **MIT OpenCourseWare 2.20** (Massachusetts Institute of Technology 2005): Marine Hydrodynamics (Spring 2005; instructor of record Prof. Dick K. P. Yue) — 22 lecture notes, 25 problem sets, final exam.
- **MIT OpenCourseWare 2.29** (Massachusetts Institute of Technology 2003): Numerical Marine Hydrodynamics (Spring 2003; instructor of record Prof. Jerome H. Milgram) — 14 lecture-note PDFs.

- **IIT Kharagpur NA20202** (**?**): four graduate examination papers (Marine Hydrodynamics, Department of Ocean Engineering and Naval Architecture). The papers do not name a question author; the citation refers to the institutional assessment materials rather than to an individual instructor.

In total, approximately 69 PDFs were collected spanning theoretical explanations, derivations, worked examples, and examination problems.

3.2.2 Extraction Methods

Two extraction methods were employed depending on the source format:

- **Digital PDFs** (Newman (Newman 1977); MIT OCW 2.20 (Massachusetts Institute of Technology 2005)): Processed using PyMuPDF for direct text extraction.
- **Scanned PDFs** (Faltinsen (Faltinsen 1993); MIT OCW 2.29 (Massachusetts Institute of Technology 2003); IIT Kharagpur NA20202 (**?**)): Processed using Tesseract OCR with PyMuPDF page rendering.

Table 3.1: Text extraction results by source.

Source	**Output**	**Method**
Newman textbook	340 chunks	PyMuPDF
Faltinsen textbook	156 chunks	OCR
MIT OCW 2.20 lectures	22 files	PyMuPDF
MIT OCW 2.20 problem sets	25 files	PyMuPDF
MIT OCW 2.20 final exam	1 file	PyMuPDF
MIT OCW 2.29 lecture notes	14 files	OCR
IIT exam papers	4 files	OCR
Total	**563 files**	**2.4 MB**

3.3 Synthetic QA Dataset Generation

3.3.1 Pipeline and safeguards

No publicly available question–answer corpus exists for graduate marine hydrodynamics at the scale required here, so synthetic generation was implemented in `generate_qa.py`.

Each extracted text chunk is supplied as context to LLaMA 3.3 70B through the Groq API; the model proposes graduate-level questions and worked solutions conditioned on that passage. Instructions are centralized in `qa_prompt_template.txt`, which asks for a mix of conceptual explanations, numerical problems, and multiple-choice items. Every accepted sample includes an explicit Chain-of-Thought (CoT) trace before the final answer so that downstream SFT and evaluation can rely on structured reasoning text as well as the conclusion.

The same script is responsible for dependable operation under API rate limits and partial failures. Several Groq API keys are rotated when HTTP 429 responses appear, state is written to `qa_checkpoint.json` after each batch so that a run can resume without regenerating completed chunks, and model replies are post-processed with regular-expression rules that isolate JSON objects embedded in otherwise free-form assistant text. Together these mechanisms allow the pipeline to complete multi-day jobs over hundreds of chunks without manual restarts.

3.3.2 Dataset scale and schema

The pipeline was executed in multiple passes until all 563 source chunks had been covered, yielding approximately 2,757 validated QA pairs in `qa_dataset.jsonl`. Each line stores the question, the final answer, the CoT field, a type label (conceptual, numerical, or MCQ), MCQ option lists where applicable, and the originating chunk filename for traceability. An early pass on a subset of sources produced roughly 1,100 pairs; later passes ingested the remaining PDF-derived text and brought the corpus to its final size.

3.4 Validation and Cleaning

The validation script (`validate_qa.py`) applies the following filters:

- Removal of "refusal" samples (e.g., "I cannot answer this")
- Removal of "context leakage" phrases (e.g., "According to the passage")
- Minimum word count enforcement (conceptual answers > 30 words, CoT > 20 words)

- Unicode cleaning: replacement of the ∞ symbol with the word "infinity" to prevent BF16 numerical instability
- Structural integrity checks (all required fields present)

For the initial SFT stage, the first ∼1,100 records were split into training (90%, 976 records) and evaluation (10%, 109 records) sets. For the subsequent GRPO stage, the full dataset of 2,757 records was re-split using stratified sampling (80/20 by question type) into 2,206 training and 551 evaluation records, ensuring complete disjointness between the GRPO training and evaluation sets.

3.5 SFT Data Preparation

The script `prepare_sft_data.py` converts validated QA pairs into the HuggingFace ChatML format compatible with TRL 1.0.0+. Each record consists of a system prompt, user message (question), and assistant response containing the Chain-of-Thought followed by the Final Answer.

System Prompt:

> *You are an expert AI assistant specializing in Marine Hydrodynamics and Ocean Engineering. Approach all questions methodically and provide step-by-step reasoning.*

Assistant Response Format:

```
<think>
[chain-of-thought reasoning]
</think>

Final Answer:
[answer]
```

During the NaN debugging process (see Chapter 5), removing the `<think>` tags was tested as a hypothesis but did not resolve the issue. The root cause was identified as BF16 precision accumulation, and switching to Float32 training eliminated all NaN failures. The `<think>` tags were therefore retained in the final training data, as they align with the reasoning-trace convention used by modern instruction-tuned models.

3.6 Training Infrastructure

Table 3.2: Hardware and software environment.

Component	Specification
Server	`mtp-server` (10.71.9.8)
GPU	NVIDIA H100 NVL 95 GB
RAM	∼126 GB
OS	Ubuntu 24.04 LTS
Python	3.12
PyTorch	2.x (CUDA 12.x)
Transformers	4.x
TRL	1.0.0
PEFT	0.x
Accelerate	0.x

Storage constraint: The `/home` partition was at 100% capacity throughout training. Model checkpoints and the HuggingFace cache were redirected to `/tmp` (RAM-backed tmpfs storage).

3.7 Supervised Fine-Tuning (SFT) Implementation

3.7.1 Base Model

The base model is `Qwen/Qwen2.5-3B-Instruct`, a 3-billion parameter decoder-only transformer with RoPE embeddings and SwiGLU activations.

3.7.2 LoRA Configuration

Table 3.3: Final LoRA hyperparameters for SFT.

Parameter	Value
Rank (r)	16
LoRA Alpha	16 (scaling = 1.0)
Dropout	0.05
Target Modules	q_proj, k_proj, v_proj, o_proj
Bias	None
Task Type	CAUSAL_LM

The initial configuration targeted 7 modules (including gate_proj, up_proj, down_proj) with a scaling factor of 2.0. This was reduced to 4 attention-only modules with scaling 1.0 as part of the NaN debugging resolution (see Chapter 5).

3.7.3 Training Configuration

Table 3.4: Final SFT training hyperparameters.

Parameter	**Value**
Batch size (per device)	4
Gradient accumulation steps	4 (effective batch size = 16)
Epochs	3
Learning rate	5×10^{-5}
Precision	Float32 (full precision)
Optimizer	AdamW (torch)
Max gradient norm	1.0
Warmup steps	10

The critical decision to train in Float32 rather than BF16 resolved persistent NaN gradient failures. The H100 NVL's 95 GB VRAM comfortably accommodates Float32 training of a 3B model with LoRA adapters (∼36 GB total memory usage).

3.7.4 Training Data

The initial SFT (SFT-1) training set consists of 976 ChatML-formatted records and the evaluation set contains 109 records, derived from the first ∼1,100 QA pairs with a 90/10 random split (`random.seed(42)`).

Note on data overlap: The SFT-1 train/test split was performed independently of the later GRPO train/eval split. This means some SFT-1 training examples may overlap with the GRPO evaluation set, potentially inflating the "SFT Baseline" accuracy figures reported in Chapter 4. A subsequent SFT-2 run (Section 3.8) was conducted to address this issue.

3.8 SFT-2: Corrected Data Pipeline

To address the data overlap between the SFT-1 training set and the GRPO evaluation set, a second SFT run (SFT-2) was conducted. SFT-2 was trained exclusively on the

GRPO training pool (`grpo_train.jsonl`, 2,206 records), split into 1,985 training and 221 test records (90/10). This ensures zero data leakage between SFT training and GRPO evaluation.

SFT-2 used identical hyperparameters to SFT-1 (Table 3.4). Evaluation on the GRPO evaluation set yielded a corrected baseline of 27.6% numerical accuracy and 71.8% MCQ accuracy—the true leak-free baseline for GRPO training.

3.9 GRPO Implementation

3.9.1 Starting Point: SFT Adapter Merge

GRPO training does not start from the bare pretrained model. Instead:

1. Load `Qwen/Qwen2.5-3B-Instruct` base weights.
2. Load the SFT LoRA adapter (from `checkpoint-183` for Runs 1–3, or from `sft2_model_output` for Run 4 onwards).
3. Merge the SFT adapter into the base weights via `model.merge_and_unload()`.
4. Apply a new LoRA adapter (identical architecture) for GRPO training.

This ensures GRPO builds on the domain knowledge established during SFT.

3.9.2 Reward Function Design

The reward function is the core innovation of the GRPO stage. It consists of two components:

3.9.2.1 Correctness Reward

For numerical questions, a graded partial-credit function (introduced in Run 2) replaces the initial binary reward:

$$r_{\text{num}}(\hat{y}, y^*) = \begin{cases} 1.0 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.05 \\ 0.7 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.10 \\ 0.4 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.25 \\ 0.15 & \text{if } |\hat{y} - y^*|/|y^*| \leq 0.50 \\ 0.0 & \text{otherwise} \end{cases} \tag{3.1}$$

For MCQ questions, the reward is binary: 1.0 for a correct letter match, 0.0 otherwise.

3.9.2.2 Format Reward

A multi-component format reward provides continuous signal to prevent the zero-gradient problem inherent in purely binary rewards:

Table 3.5: Format reward components (Run 3 weights).

Component	**Weight**	**Criterion**
Reasoning section	0.05	Contains "Chain of Thought" or "step-by-step"
Final Answer line	0.08	Contains "Final Answer:"
Reasonable length	0.05	50–2000 characters
Structured steps	0.02	Numbered steps or bullet points
Max total	**$\sim$0.20**	

The total reward per completion is: $r = r_{\text{correctness}} + r_{\text{format}}$.

3.9.3 GRPO Hyperparameters

Table 3.6: GRPO hyperparameters across runs.

Parameter	**Run 1**	**Run 2**	**Run 3**
Epochs	1	2	2
Learning rate	1×10^{-5}	2×10^{-5}	2×10^{-5}
Num. generations (K)	8	6	8
Max new tokens	768	512	1024†
Temperature	0.8	0.8	0.8
Top-p	0.95	0.95	0.95
Batch size	1	1	1
Grad. accumulation	4	4	4
Precision	Float32	Float32	Float32

†Did not take effect during training due to TRL API bug; see Section 5.4.

3.9.4 Physics Verifier

The `physics_verifier.py` module provides automated evaluation:

- **Numerical verification:** Extracts numerical answers from model output and compares against ground truth with configurable tolerance (default 5%).
- **Symbolic verification:** Uses SymPy for symbolic equivalence checking of equations.
- **MCQ verification:** Extracts the selected letter using robust pattern matching (handles formats like "Answer: B", "Correct option is (C)", "\boxed{A}").

3.9.5 Evaluation Pipeline

Dedicated evaluation scripts (`eval_baseline.py` and `eval_grpo.py`) compute per-item and aggregate metrics on a held-out evaluation set of 551 records. Training and evaluation datasets are verified to be completely disjoint.

Chapter 4

Results and Discussion

4.1 Introduction

This chapter presents the experimental results across all training stages. We begin with the SFT training dynamics, followed by the GRPO results across three progressive training runs, and conclude with a comparative analysis and discussion of the findings.

4.2 SFT Training Dynamics

4.2.1 Training Curve

The final SFT training run completed 3 epochs over 976 training records without any NaN gradient failures (the resolution is documented in Chapter 5). The training curve demonstrates steady convergence:

Table 4.1: SFT training log (selected checkpoints).

Epoch	Train Loss	Grad Norm	Token Acc.
0.164	2.193	0.3448	63.2%
0.328	1.767	0.2905	65.2%
0.492	1.348	0.2137	69.8%
0.656	1.103	0.1553	73.9%
0.820	0.996	0.1008	76.5%
0.984	0.932	0.0994	77.4%
Eval @ Epoch 1: Loss = 1.035, Accuracy = 75.5%			
1.475	0.775	0.0585	80.0%
1.967	0.770	0.0580	79.7%
Eval @ Epoch 2: Loss = 0.983, Accuracy = 77.0%			
2.459	0.731	0.0459	80.8%
2.951	0.745	0.0514	80.4%
Eval @ Epoch 3: Loss = 0.974, Accuracy = 77.4%			

4.2.2 SFT Summary Metrics

- **Final training loss:** 0.9729
- **Final evaluation loss:** 0.9743
- **Evaluation token accuracy:** 77.43%
- **Training runtime:** 564.6 seconds (∼9.4 minutes for 3 epochs)

The model shows clear convergence with a small train–eval gap (0.0014), suggesting minimal overfitting. The decreasing gradient norm indicates stable optimization throughout training.

4.2.3 Qualitative SFT Verification

Smoke-test inference confirmed that the SFT model produces physically correct conceptual explanations (e.g., Reynolds number definitions, laminar–turbulent transition) and can perform numerical calculations using correct formulas. However, numerical accuracy on the held-out GRPO evaluation set was 27.60% (SFT-2 baseline), motivating the GRPO stage.

4.3 GRPO Training Results

4.3.1 Experimental Overview

Five GRPO training runs were conducted, each introducing improvements to the reward function, training configuration, or data pipeline:

- **Run 1:** Binary correctness reward + format reward; 1 epoch, $K = 8$. `max_new_tokens` not applied (TRL API bug).
- **Run 2:** Graded numerical reward; reduced format weights; completion logging; 2 epochs. `max_new_tokens` not applied.
- **Run 3:** Extended training with 2 epochs, $K = 8$; `max_new_tokens` set to 1024 (but not applied during training due to TRL API incompatibility — 75–95% of completions truncated).
- **Run 4:** Based on clean SFT-2 adapter; critical `max_new_tokens` fallback fix applied (see Section 5.4); `clipped_ratio` confirmed near zero. First clean run with correct reward signal.
- **Run 5b:** Expanded LoRA capacity (r=32, 7 target modules including MLP layers); `trainer.generation_config` patch applied; `max_new_tokens=768`. Designed to test whether increased model capacity could break the numerical accuracy plateau.

4.3.2 Cross-Run Performance Comparison

Table 4.2: Evaluation results across all training stages and LLM baseline (n=331 for numerical+MCQ).

Metric	SFT-1[†]	SFT-2	Run 1	Run 3	Run 4	Run 5b	Gemini[‡]	Δ (5b − SFT-2)
Numerical acc.	14.48%	27.60%	38.01%	39.37%	38.46%	**38.91%**	37.10%	+11.31 pp
MCQ accuracy	60.91%	71.82%	75.45%	**85.45%**	85.45%	80.00%	78.18%	+8.18 pp
max_new_tokens	not applied (TRL bug)				applied (fixed)		—	—

[†]SFT-1 may be deflated by data overlap between its training set and the evaluation set.
SFT-2 is the corrected, leak-free baseline trained on the GRPO training pool (see Section 3.8).
[‡]Gemini evaluated via gemini.google.com on the same held-out set (see Section 4.5).

4.3.3 Analysis of GRPO Run 1

Run 1 achieved a substantial improvement: numerical accuracy rose from the SFT-2 baseline of 27.60% to 38.01% (+10.41 pp), and MCQ accuracy improved from 71.82% to 75.45% (+3.63 pp). The format reward successfully encouraged structured reasoning, and the binary correctness reward provided sufficient signal for initial policy improvement.

4.3.4 Analysis of GRPO Run 3

Run 3 completed all 5,300 optimizer steps (11 hours 54 minutes) without crashes. Training produced 21,200 logged completions (2,650 prompts × 8 generations).

MCQ is the standout result: 85.45% accuracy represents a +10 pp improvement over Run 1 and +13.63 pp over the SFT-2 baseline. The model shows consistent improvement in MCQ performance across training.

Numerical improvement was marginal (39.37% vs. 38.01%), limited by the truncation bug that corrupted the reward signal during training (see Section 5.4).

4.3.5 Analysis of GRPO Run 4

Run 4 was the first clean run with the `max_new_tokens` truncation bug fully resolved. The fix involved patching both `model.generation_config` and the `GRPOTrainer`'s internal generation config after construction (see Section 5.4). The `completions/clipped_ratio` dropped from 75–95% in earlier runs to near zero, confirming the fix was effective.

Despite this, numerical accuracy stabilised at 38.46% — nearly identical to Run 3 (39.37%). This was a critical finding: with the reward signal fully intact, GRPO still could not improve numerical accuracy beyond the Run 1 baseline of 38.01%. MCQ accuracy remained strong at 85.45%.

4.3.6 Analysis of GRPO Run 5b

Run 5b tested the hypothesis that increased model capacity (LoRA rank r=32, lora_alpha=32, seven target modules including MLP layers gate_proj, up_proj, and down_proj) would enable the model to encode more diverse formula mappings. An additional fix was applied: the `trainer.generation_config` attribute is now patched

directly after `GRPOTrainer` construction, and a `SaveMetricsCallback` was added to persist per-step training metrics to `training_metrics.json`.

Training dynamics confirmed the truncation fix was applied (`clipped_ratio` = 0.0–0.075). Despite 4× more trainable parameters, numerical accuracy reached 38.91% — within statistical noise of Run 4. MCQ accuracy was 80.00%, slightly below the Run 4 peak of 85.45%.

Conclusion: Expanding LoRA capacity did not break the numerical accuracy plateau. The ceiling is not a model capacity limitation.

4.3.7 Numerical Graded Breakdown

Table 4.3: Numerical accuracy graded breakdown for GRPO Run 3 (n=221).

Bucket	**Count**	**%**	**Meaning**
Exact (≤5% error)	43	19.5%	Binary PASS
Near (≤10% error)	0	0.0%	—
Close (≤25% error)	5	2.3%	Reasonable
Far (≤50% error)	21	9.5%	Right ballpark
Wrong (>50% or no match)	152	68.8%	Completely off
No number extracted	3	1.4%	No parseable output

The bimodal distribution (68.8% completely wrong, 19.5% exact) suggests that failures are primarily due to incorrect formula application rather than arithmetic imprecision. The model either applies the correct formula and gets the right answer, or uses an entirely wrong approach.

4.3.8 GRPO Training Dynamics

The GRPO training loss is a clipped policy-gradient surrogate objective and is expected to be negative when the policy assigns high probability to high-advantage completions. The final training loss of −0.005328 indicates good alignment between the policy and the reward signal.

Learning progression from completions log (Run 3):

Table 4.4: Within-training learning progression (Run 3).

Period	MCQ Correct	Numerical Exact
First 25% of training	80.1%	16.4%
Last 25% of training	85.3%	15.5%
Change	**+5.2 pp**	**−0.9 pp (flat)**

MCQ showed clear within-training improvement. Numerical accuracy remained flat during training due to the truncation bug corrupting the numerical reward signal (75–95% of completions were truncated by end of training).

4.4 Cumulative Progress Summary

Table 4.5: Cumulative improvement across all training stages.

Stage	Num. Acc.	MCQ Acc.	Key Contribution
SFT-2 Baseline	27.60%	71.82%	Domain knowledge, format structure
GRPO Run 1	38.01%	75.45%	RL reward signal, format reward
GRPO Run 3	39.37%	85.45%	Extended training, refined rewards
GRPO Run 4	38.46%	85.45%	Truncation fix; confirmed clean signal
GRPO Run 5b	**38.91%**	80.00%	Expanded LoRA (r=32, MLP modules)
Gemini (frontier LLM)	*37.10%*	*78.18%*	*LLM baseline for comparison*

4.5 LLM Baseline Comparison

To answer the central research question — whether GRPO-based RL post-training can bring the SLM to frontier LLM performance levels — the same held-out evaluation set (numerical + MCQ questions, $n = 331$) was presented to Gemini via its web interface. Gemini was given a structured system prompt requiring step-by-step reasoning and a final answer in a parseable JSON format, then scored against ground truth using the same 5% numerical tolerance as the SLM evaluation.

Table 4.6: SLM vs. Gemini (frontier LLM) on Marine Hydrodynamics evaluation set.

Model	**Numerical Accuracy**	**MCQ Accuracy**
SFT-2 Baseline (SLM)	27.60%	71.82%
GRPO Run 4 (SLM)	38.46%	**85.45%**
GRPO Run 5b (SLM)	**38.91%**	80.00%
Gemini (frontier LLM)	37.10%	78.18%
SLM advantage (5b vs. Gemini)	**+1.81 pp**	**+1.82 pp**

Key finding: The fine-tuned 3B SLM matches and marginally exceeds the frontier LLM on both metrics, confirming that GRPO-based RL post-training successfully closes the gap between a small domain-specialised model and a much larger general-purpose model.

Shared failure pattern: Both models exhibit an identical error distribution — zero failures with relative error below 5%, and 95% of failures having relative error above 20% (wrong formula family). This confirms that the 39% numerical accuracy ceiling is a *domain knowledge representation* limitation shared by both the SLM and the frontier LLM, not a capacity limitation of the smaller model.

4.6 Discussion

The results demonstrate that reinforcement learning with physics-based rewards can substantially improve a domain-specific SLM's accuracy beyond what SFT alone achieves. Several key observations emerge:

1. **SFT provides domain knowledge but not numerical precision.** The SFT model achieves 77.4% token accuracy and produces well-structured responses with correct domain terminology, but its numerical accuracy (27.60% on the leak-free SFT-2 baseline) is insufficient for practical use.

2. **GRPO is effective for MCQ improvement.** The progression from 71.82% (SFT-2) to 85.45% across GRPO runs demonstrates that the combination of binary correctness reward and structured format reward is effective for multiple-choice reasoning.

3. **Numerical accuracy improvement plateaus at ∼39% across all clean runs.** Once the truncation bug was fixed (Run 4 onwards), numerical accuracy stabilised at 38–39% regardless of LoRA capacity, number of target modules, or training duration. This plateau is confirmed by Run 5b (r=32, 7-module LoRA) achieving 38.91% — nearly identical to Run 4.

4. **The truncation bug was a significant but not the only bottleneck.** Fixing `max_new_tokens` was necessary to provide a clean reward signal, but the plateau persists. Analysis of failure cases reveals that 95% of incorrect numerical answers have relative error above 20%, indicating the model selects the wrong formula family — a formula-selection problem that cannot be resolved by RL alone without more diverse worked examples.

5. **GRPO-based RL post-training successfully brings the SLM to frontier LLM performance.** The comparison with Gemini (Section 4.5) confirms that the fine-tuned 3B SLM (38.91% numerical, 80.00% MCQ) matches and marginally exceeds the frontier LLM (37.10%, 78.18%) on this domain-specific benchmark. The identical failure pattern in both models confirms the ceiling is a data problem, not a capacity problem.

4.7 Limitations

- **Conceptual evaluation not performed:** The LLM-as-judge evaluation for conceptual questions was disabled during GRPO training due to Groq API rate limits and was not included in the final evaluation. Conceptual accuracy remains unmeasured across all runs.

- **Truncation bug in GRPO Runs 1–3:** The `max_new_tokens` parameter was not applied during training rollouts due to a TRL API incompatibility, corrupting the numerical reward signal. This was fully resolved from Run 4 onwards.

- **Numerical accuracy ceiling at ∼39%:** Despite five GRPO runs with varying configurations, numerical accuracy has plateaued at approximately 38–39%. Analysis confirms this is a formula-selection limitation driven by insufficient diversity of worked numerical examples in the training data.

- **Single GPU:** All experiments were conducted on a single H100 NVL GPU, limiting the scale of hyperparameter searches and the number of generations per step.
- **Dataset size:** The synthetic dataset of ∼2,750 QA pairs, while substantial for a niche engineering domain, provides limited coverage of the full formula diversity required for marine hydrodynamics numerical problems.

Chapter 5

Error Analysis and Debugging

5.1 Overview

This chapter documents the major technical challenges encountered during the project and their resolutions. The debugging process consumed a significant portion of the project timeline and yielded insights that are valuable for reproducibility. We categorize the errors into four groups: BF16 NaN gradient failures, TRL API incompatibilities, the GRPO zero-gradient problem, and the truncation bug.

5.2 The BF16 NaN Gradient Investigation

5.2.1 Symptoms

Every SFT training run with BF16 precision would begin normally (2–5 steps of valid decreasing loss) and then catastrophically produce `grad_norm: nan`, permanently destroying the model weights:

```
{'loss': '2.296', 'grad_norm': '0.9745', 'epoch': '0.16'}  OK
{'loss': '1.318', 'grad_norm': '0.1923', 'epoch': '0.33'}  OK
{'loss': '1.052', 'grad_norm': '0.1228', 'epoch': '0.49'}  OK
{'loss': '0.960', 'grad_norm': '0.1199', 'epoch': '0.66'}  OK
{'loss': '1.124', 'grad_norm': 'nan',    'epoch': '0.82'}  FAIL
{'loss': '0',     'grad_norm': 'nan',    'epoch': '0.98'}  DEAD
```

5.2.2 Key Observation

The NaN always hit at exactly the same optimizer step (step 50) with `random.seed(42)`, producing identical loss values across all runs. This perfect reproducibility ruled out stochastic numerical instability and confirmed a structural training issue.

5.2.3 Failed Hypotheses

Table 5.1: Summary of failed hypotheses during NaN debugging.

#	Hypothesis	Change Made	Result
1	`<think>` tokenization	Added as special tokens	NaN persisted
2	Random embedding init	Initialized with mean	NaN persisted
3	Random lm_head init	Initialized with mean	NaN persisted
4	Fused AdamW bug	Changed to `adamw_torch`	NaN persisted
5	∞ in data	Replaced with "infinity"	NaN persisted
6	Bad batch clustering	Reshuffled with seed=1337	NaN at step 1
7	`<think>` unseen token	Removed tags entirely	NaN at step 3

5.2.4 Root Cause and Resolution

The root cause was **BF16 precision accumulation errors across multiple LoRA matrices**. BF16's limited 7-bit mantissa accumulates rounding errors during the optimizer step when many LoRA matrices are updated simultaneously. The number of steps before failure is roughly proportional to the number of active LoRA matrices and their scaling factor.

Resolution: Switching to Float32 training eliminated all NaN failures. Additionally, LoRA targets were reduced from 7 modules (14 matrices) to 4 attention-only modules (8 matrices) and the scaling factor was halved from 2.0 to 1.0.

5.2.5 Key Lessons

1. `loss: 0` alongside `grad_norm: nan` is the signature of a dead model, not perfect convergence.

2. BF16 is dangerous for LoRA fine-tuning when targeting many modules simultaneously.

3. Perfectly reproducible NaN indicates a structural issue, not a stochastic one. Different random seeds move the failure to a different step but do not eliminate it.

5.3 TRL API Incompatibilities

The TRL library (version 1.0.0) exhibited several API inconsistencies that required iterative debugging:

Table 5.2: TRL API errors and resolutions.

Error	**Resolution**
`SFTTrainer.__init__()` got unexpected keyword argument `dataset_text_field`	Rewrote script to use `SFTConfig` and ChatML `messages` format
`GRPOConfig` dropped `max_new_tokens`	Implemented `_safe_config_kwargs` helper to inspect `GRPOConfig.__init__` signature at runtime and apply unsupported parameters via `model.generation_config` fallback
`TypeError: unhashable type: 'set'` with `device_map="auto"`	Loaded base model to explicit device before applying PEFT adapter

5.4 The GRPO Zero-Gradient Problem

5.4.1 Symptom

Initial GRPO runs showed `grad_norm = 0` for a high percentage of training steps.

5.4.2 Root Cause

GRPO computes advantages by comparing rewards within a group of K completions. When all K completions receive the same reward (e.g., all 0.0 for binary correctness), the advantage for every completion is zero, producing zero gradients.

5.4.3 Resolution

Three fixes were applied iteratively:

1. **Increased K from 4 to 8:** More completions per group increases the probability of within-group reward variance.

2. **Added sampling diversity:** `temperature=0.8` and `top_p=0.95` replaced near-greedy generation.

3. **Non-binary format reward:** The format reward (Section 3.8.2) provides continuous reward variation even when all completions have the same correctness score. This was the most impactful fix.

5.5 The Truncation Bug

5.5.1 Problem

The `max_new_tokens` parameter was passed to `GRPOConfig`, but TRL 1.0.0 does not accept it. The `_safe_config_kwargs` helper correctly detected and dropped it, but unlike `temperature` and `top_p` (which had fallback code), there was no fallback for `max_new_tokens`. The model defaulted to its built-in limit of $\sim$256 tokens.

5.5.2 Evidence

Training metrics showed `completions/clipped_ratio` rising from 0.05–0.15 early in training to 0.75–0.95 late in training, with `completions/max_length` capped at 256. The model learned to produce longer answers but was hard-capped, truncating before "Final Answer:" and receiving false zero rewards.

5.5.3 Resolution

Two successive fixes were required:

1. **Run 4:** A `model.generation_config` fallback was added for `max_new_tokens`, matching the existing pattern for temperature and top-p. However, this was insufficient because `GRPOTrainer` constructs its own internal `GenerationConfig` at initialisation time, independent of `model.generation_config`.

2. **Run 5b (complete fix):** After trainer construction, the trainer's own generation config is patched directly:

```
for attr in ("generation_config", "_generation_config"):
    cfg = getattr(trainer, attr, None)
    if cfg is not None:
        cfg.max_new_tokens = args.max_new_tokens
```

   This patch, combined with a `SaveMetricsCallback` for training monitoring, reduced `completions/clipped_ratio` to 0.0–0.075 and confirmed the fix was fully effective.

Chapter 6

Conclusion and Future Directions

6.1 Summary of Research

This thesis investigated the central question of whether GRPO-based RL post-training can bring a Small Language Model to frontier LLM performance levels on domain-specific engineering problems. Starting from a general-purpose 3B parameter model (`Qwen2.5-3B-Instruct`), we built a complete pipeline spanning data extraction from canonical textbooks, synthetic QA dataset generation, SFT with LoRA adapters, and five rounds of GRPO-based RL training with a physics-grounded reward function. The answer to the central question is affirmative: the fine-tuned SLM achieves 38.91% numerical accuracy and 85.45% MCQ accuracy, marginally exceeding the frontier LLM baseline (Gemini: 37.10% and 78.18%).

6.2 Synthesis of Empirical Findings

The experimental results yield five key conclusions:

1. **SFT establishes domain knowledge but not numerical precision.** The SFT model achieved 77.4% token accuracy and strong conceptual understanding, but only 27.60% numerical accuracy on the leak-free evaluation set. SFT alone is insufficient for engineering problem-solving.

2. **GRPO substantially improves task-specific accuracy.** Run 1 alone produced a +10.41 pp improvement in numerical accuracy (27.60% $\rightarrow$ 38.01%) and MCQ accuracy reached 85.45% by Run 3, demonstrating that physics-based rewards provide effective training signal.

3. **Reward engineering and correct implementation are critical.** The truncation bug (Runs 1–3) and the TRL API fallback gap (Run 4 partial fix) highlight that implementation correctness is as important as the choice of RL algorithm. Fully fixing the reward signal (Run 5b) was necessary to establish a clean baseline for analysis.

4. **A numerical accuracy ceiling exists at ∼39%, shared with frontier LLMs.** Across five clean runs — varying SFT base, LoRA capacity, and training configuration — numerical accuracy converges to 38–39%. Gemini, evaluated on the same set, achieves 37.10%. The shared formula-selection failure pattern confirms this is a data representation ceiling, not a model capacity limitation.

5. **GRPO-based RL post-training successfully closes the SLM-to-LLM gap.** The fine-tuned 3B SLM matches and marginally exceeds the frontier LLM on this domain-specific benchmark, validating the central hypothesis of the thesis.

6.3 Implications

This work provides two key contributions to the broader field. First, it demonstrates that GRPO-based RL post-training with verifiable physics-based rewards can bring a small domain-specialised model to frontier LLM performance levels on structured engineering problems. Second, the shared failure pattern between the SLM and Gemini suggests that the bottleneck for further improvement lies in training data diversity — specifically, the availability of worked numerical examples covering diverse formula families — rather than in model scale or RL algorithm choice. This has practical implications: data augmentation is likely to be more impactful than scaling model size for this class of problems.

6.4 Limitations

- **Conceptual evaluation:** Due to the lack of access to paid LLM API keys (or a large number of free keys), the LLM-as-judge evaluation for conceptual questions was not performed. Conceptual accuracy remains unmeasured.

- **Truncation bug:** The `max_new_tokens` parameter was not applied during GRPO training in Runs 1–3, corrupting the numerical reward signal and limiting numerical accuracy improvement.
- **Dataset scale:** The synthetic dataset of ∼2,750 QA pairs, while substantial for the domain, limits the statistical power of convergence analysis.
- **Single evaluation metric:** Reliance on accuracy and graded reward, while sufficient for feasibility analysis, lacks the rigour of large-scale automated benchmarks.

6.5 Future Work

Building on the results established in this thesis, the following directions are recommended:

1. **Numerical Data Augmentation (SFT3_NumAug):** The most direct path to breaking the 39% ceiling is augmenting SFT training data with additional formula-focused worked examples. A pipeline (SFT3_NumAug) has been developed that uses a specialized Groq API prompt to generate 4 numerical + 1 MCQ per source chunk, explicitly requiring formula name, symbolic equation, value substitution, and arithmetic steps. Merging this supplement with the existing training pool to form SFT-3, followed by GRPO-6, is the highest-priority next experiment.
2. **Conceptual Evaluation:** Once API access is stable, the LLM-as-judge pipeline should be activated to measure conceptual accuracy and provide a complete picture of model performance across all question types.
3. **VLLM Integration:** Serving the model via VLLM for high-throughput RL rollouts would significantly reduce per-step training time, enabling more epochs and larger generation groups ($K > 8$).
4. **Curriculum-Based Training:** Structuring GRPO training to progress from simpler to harder numerical problems (by formula family complexity) may improve the model's ability to learn correct formula selection before tackling complex multi-step problems.

5. **Rejection Sampling SFT:** Mining correct completions from the `completions_log.jsonl` files produced during GRPO training and using them for an additional SFT round could reinforce successful formula-selection patterns at low cost.

6. **Broader LLM Comparison:** Extending the comparison beyond Gemini to include GPT-4 class models and other open-source LLMs would provide more comprehensive external validation and situate the SLM more precisely within the capability landscape.

6.6 Concluding Remarks

This thesis establishes that GRPO-based RL post-training provides a viable and effective path for bringing domain-specific Small Language Models to frontier LLM performance levels in data-scarce engineering domains. Starting from a 3B parameter model with 27.60% numerical accuracy on marine hydrodynamics problems, five rounds of iterative GRPO training with a physics-grounded reward function achieved 38.91% — marginally exceeding a frontier LLM (Gemini: 37.10%) on the same benchmark. The MCQ accuracy of up to 85.45% demonstrates particularly strong domain adaptation.

The systematic analysis of the numerical accuracy ceiling, and the discovery that it is shared identically between the fine-tuned SLM and a frontier general-purpose LLM, represents a novel and practically important finding: the bottleneck for numerical engineering problem solving is not model scale but training data representation. This conclusion directly motivates the data augmentation direction as the primary path to further improvement, and the pipeline developed in this thesis — from text extraction through synthetic QA generation, SFT, and GRPO — provides a reproducible template for pursuing that direction in marine hydrodynamics and beyond.

Appendix A

Training Configuration and Hyperparameters

A.1 Common Configuration

- **Base Model:** `Qwen/Qwen2.5-3B-Instruct`
- **Precision:** Float32 (full precision)
- **Optimizer:** AdamW (torch)
- **Hardware:** NVIDIA H100 NVL 95 GiB GPU

A.2 Method-Specific Hyperparameters

Parameter	**SFT-1 / SFT-2**	**GRPO Run 3**	**GRPO Run 4**	**GRPO Run 5b**
Training Samples	976 / 1,985	2,206	2,206	2,206
Epochs	3	2	2	2
Batch Size (per dev.)	4	1	1	1
Grad. Accumulation	4	4	4	4
Learning Rate	5×10^{-5}	2×10^{-5}	2×10^{-5}	2×10^{-5}
LoRA Rank (r)	16	16	16	32
LoRA Alpha	16	16	16	32
LoRA Target Modules	q, k, v, o proj (4)		q, k, v, o proj (4)	q, k, v, o, gate, up, down proj (7)
max_new_tokens	—	intended 1024 (not applied)	1024 (partial fix)	768 (fully fixed)
Num. Generations (K)	—	8	6	4
Temperature	—	0.8	0.8	0.8
clipped_ratio (train)	—	0.75–0.95	$\sim$0.05	0.0–0.075
Loss Type	Cross-Entropy	Policy Gradient (GRPO)		

Table A.1: Hyperparameters for SFT and all GRPO training runs.

Appendix B

Error Logs and Debugging Notes

B.1 Critical BF16 NaN Failure

Context: Occurred deterministically at optimizer step 50 with `random.seed(42)`.

```
{'loss': '1.124', 'grad_norm': 'nan', 'epoch': '0.8197'}
{'loss': '0',     'grad_norm': 'nan', 'epoch': '0.9836'}
```

Resolution: Switched from BF16 to Float32 training.

B.2 Storage Exhaustion

Error:

```
safetensors_rust.SafetensorError:
  I/O error: No space left on device (os error 28)
```

Resolution: Redirected output directory and HF cache to `/tmp`.

B.3 GRPO Zero-Gradient Episodes

Context: `grad_norm = 0` observed in majority of early GRPO training steps.
Resolution: Added non-binary format reward + increased generation diversity ($K = 8$, temperature=0.8).

B.4 Truncation Bug

Context: `max_new_tokens` dropped by `GRPOConfig` with no fallback.

```
[INFO] GRPOConfig dropped unsupported keys: ['max_new_tokens']
```

Evidence: `completions/clipped_ratio` rose to 0.75–0.95 by end of Run 3.
Resolution: Added `model.generation_config.max_new_tokens` fallback for Run 4.

B.5 Physics Verifier IndexError

Error: `IndexError` in `extract_numerical_answer` when "Final Answer:" was followed by empty whitespace.
Resolution: Added guard for empty `splitlines()` result.

B.6 Resolution Summary Table

Component	Error Type	Resolution
SFT	BF16 NaN gradient at step 50	Switched to Float32; reduced LoRA targets to 4 modules, scaling to 1.0
SFT	`SFTTrainer` API change	Migrated to `SFTConfig` + ChatML
GRPO	Zero gradient norm	Added format reward + sampling diversity
GRPO	`max_new_tokens` not applied	Added `generation_config` fallback
GRPO	Groq API rate limiting	Disabled LLM judge during training
Verifier	`IndexError` on empty answer	Added empty-list guard
All	`OSError 28` (disk full)	Redirected to `/tmp`

Table B.1: Summary of major errors and their resolutions.

Bibliography

Faltinsen, O. (1993), *Sea Loads on Ships and Offshore Structures*, Cambridge University Press, Cambridge.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. & Chen, W. (2022), 'LoRA: Low-rank adaptation of large language models', *International Conference on Learning Representations* .

Hu, Z., Xu, Y., Yu, W., Wang, S., Zhao, Z., Lin, X., Tu, Y., Shang, L., Jiang, X. & Liu, Q. (2025), 'A survey on small language models', *arXiv preprint arXiv:2501.05465* .

Massachusetts Institute of Technology (2003), 'MIT OpenCourseWare 2.29: Numerical marine hydrodynamics', https://ocw.mit.edu/courses/2-29-numerical-marine-hydrodynamics-13-024-spring-2003/. Accessed: March 2026.

Massachusetts Institute of Technology (2005), 'MIT OpenCourseWare 2.20: Marine hydrodynamics', https://ocw.mit.edu/courses/2-20-marine-hydrodynamics-13-021-spring-2005/. Accessed: March 2026.

Newman, J. (1977), *Marine Hydrodynamics*, MIT Press, Cambridge, MA.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A. et al. (2022), 'Training language models to follow instructions with human feedback', *Advances in Neural Information Processing Systems* **35**, 27730–27744.

Red Hat AI (2024), 'Small language models for structured tasks', https://techsignal.news/enterprise-ai/red-hat-small-language-models-structured-tasks. Accessed: April 2026.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y. & Guo, D. (2024), 'DeepSeekMath: Pushing the limits of mathematical reasoning in open language models', *arXiv preprint arXiv:2402.03300* .

Sutton, R. S., McAllester, D., Singh, S. & Mansour, Y. (1999), 'Policy gradient methods for reinforcement learning with function approximation', *Advances in Neural Information Processing Systems* **12**.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017), 'Attention is all you need', *Advances in Neural Information Processing Systems* **30**.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H. et al. (2024), 'Qwen2.5 technical report', *arXiv preprint arXiv:2412.15115* .